

Operating System Project Mid Report

1. 成员列表

姓名	学号
徐晨钧	12210126
张善	12213052

2. 设计架构

2.1 内核数据结构设计

我们为每个进程 `struct proc` 设置了一个 `struct ksignal` 结构体，封装信号处理相关信息，其定义位于 `ksignal.h` 中，具体包含：

- `sigaction_t sa[SIGMAX + 1]`：每个信号对应的处理函数及屏蔽信号集合。
- `sigset_t sigmask`：进程当前屏蔽的信号集合。
- `sigset_t sigpending`：待处理的信号集合。
- `siginfo_t siginfos[]`：用于存储信号附加信息（目前填充为0）。
- 🌟 `signal_context_stack ctx_stack`：用户上下文栈，用于支持嵌套信号处理。

设计支持最大 4 层嵌套信号处理，防止栈溢出。

2.2 系统调用接口设计

2.2.1 🌟 信号初始化与继承

- `siginit(struct proc *p)`
初始化新进程的信号状态：
 - 所有信号处理设为 `SIG_DFL`（默认处理），mask 与 pending 集合清零。
 - 初始化信号上下文栈（用于支持嵌套信号处理）。

- `siginit_fork(struct proc *parent, struct proc *child)`
在 `fork` 时调用：
 - 拷贝父进程的 `sigaction` 设置与 `sigmask`。
 - 清空 `sigpending`，防止子进程继承父进程已挂起的信号。

- `siginit_exec(struct proc *p)`
在 `exec` 时调用：
 - 重置所有非忽略（`SIG_IGN`）的信号处理函数为 `SIG_DFL`。
 - 保留原有的 mask 与 pending 集合。

2.2.2 🌟 信号处理主逻辑

- `do_signal()`

信号处理的核心函数，在每次从内核回到用户态之前被调用（如 `usertrap` 中）：

- 遍历 `sigpending` 与 `sigmask`，筛选出未被屏蔽的最高优先级信号。
- 根据信号类型执行对应动作：
 - 若为 `SIG_DFL`，调用 `setkilled()` 杀死进程。
 - 若为 `SIG_IGN`，直接返回。
 - 否则为用户定义 handler：
 1. 构造 `siginfo_t` 和 `ucontext`。
 2. 拷贝到用户栈（`copy_to_user`）。
 3. 修改 `trapframe`：
 - 设置 `epc` 指向 handler。
 - 设置参数 `a0-a2` 为信号参数。
 - 设置 `ra` 为 `sa_restorer`，使其在 handler 返回后调用 `sigreturn`。
 4. 更新 `sigmask`，添加当前信号及 handler 中设置的屏蔽信号。

该设计完全模拟真实内核的异步中断处理流程。

2.2.3 🌟系统调用实现

- `sys_sigaction(int signo, const sigaction_t *act, sigaction_t *oldact)`
 - 核心用于设置或读取某个信号的处理函数。
 - 关键逻辑包括信号合法性校验、禁止修改 `SIGKILL` 和 `SIGSTOP`。
 - 若 `act != NULL`，则用 `copy_from_user` 从用户空间拷贝新处理方式。
 - 若 `oldact != NULL`，则用 `copy_to_user` 将旧处理方式写回。
- `sys_sigkill(int pid, int signo, int code)`
 - 根据 `pid` 找到目标进程。
 - 设置目标进程对应信号为 `pending` 状态。
 - 若信号为 `SIGKILL`，直接调用 `setkilled()` 强制终止。
- `sys_sigreturn()`
 - 在信号处理函数结束后调用，恢复处理前上下文：
 - 从 `trapframe→a2` 获取用户态 `ucontext` 的地址。
 - 读取原寄存器值并恢复到 `trapframe`。
 - 恢复旧的 `sigmask`。
 - 同时从内核保存的 `signal context` 栈弹出对应帧，支持嵌套 handler。
- `sys_sigpending(sigset_t *set)`
 - 将当前进程的 `sigpending` 拷贝到用户空间，用于用户查询。
- `sys_sigprocmask(...)`
 - 目前为占位函数，未实现逻辑，保留用于后续屏蔽信号的支持。

2.2.4 用户接口简述（`signal.h`）

用户接口主要定义了信号编号、宏操作函数（如 `sigemptyset`）、以及用于 `sigaction` 与 `sigreturn` 的结构体 `sigaction_t` 与 `ucontext`。这些接口通过头文件 `signal.h` 与用户程序共享，供用户注册和管理信号使用。

3. 实现细节

1. 我们在 `ksignal` 类中添加了 `signal_context_stack` 成员类来转移 `trapframe` 的 `a2` 寄存器的地址，以避免传输过程中 `a2` 寄存器内容丢失。
2. 在 `siginit` 函数中，实现了对信号的所有内容进行初始化。
3. `siginit_fork` 用于子进程复制父进程的 `sigmask` 和 `sigactions`，并去除子进程的所有 `pending` 进程。
4. `siginit_exec` 函数重置了该进程的所有 `sigactions`。
5. 在 `do_signal` 函数中：
 - a. 函数首先找到优先级最高的 `signal`，并将其从 `sigaction` 中清除，准备处理这个 `signal`。
 - b. 处理完 `signal` 后，函数开始着手准备用户栈和上下文，并进行用户栈地址的合理性检测。用户栈地址合法时对 `trapframe` 进行临时保存。
 - c. 准备好临时 `trapframe` 内容后把用户栈地址传输给用户。
 - d. 最后设置 `trapframe`。
6. `sys_sigaction` 函数重点处理了 `SIGKILL` 与 `SIGSTOP` 信号，并保存旧信号，设置新信号。
7. 在 `sys_sigreturn` 函数中：
 - a. 函数先取出用户栈的栈顶元素，作为处理完信号（中断）后的返回地址并检查其合法性。
 - b. 检查合法后用在 `do_signal` 函数中保存的临时 `trapframe` 恢复当前 `trapframe`。
8. `sys_sigkill` 函数通过寻找对应进程，设置进程信号，并对 `SIGKILL` 指令进行及时反馈的方法对传入的信号进行处理。

4. 过程中遇到的问题

1. checkpoint4
 - a. 问题：我们遇到的问题是测试脚本卡在handler4无法继续运行。
 - b. 尝试：经过调试打印我们发现问题出现在sigreturn中tf→a2寄存器的值被改变，保存了非法地址，在地址合法性检测时无法通过因而造成卡死。
 - c. 解决方法：我们在ksignal类中新建了一个类成员last_context_addr，并在do_signal函数中设置trapframe时将其设置为user_uc_addr，避免因a2寄存器被非法修改造成的在sys_sigreturn中的卡死。
2. checkpoint6
 - a. 问题：脚本无法通过信号嵌套处理测试。

```
test basic6: [PANIC 15][src/signal-project-tests/basic.c:160]: assert error: handler6_flag ==
test basic6: [PANIC 14][src/signal-project-tests/basic.c:199]: assert_eq error: 0xFFFFFFFF
test basic6: FAILED
```

- b. 尝试：跟踪程序发现由于 `sigmask` 恢复错误信号嵌套返回后跳转异常，主流程中断。
- c. 解决方法：我们在 `struct proc` 的 `struct ksignal` 结构体中新增了 `signal_context_stack ctx_stack`：用户上下文栈，用于保存每次信号处理前的用户上下文地址（`ucontext`），从而支持多个信号处理函数的嵌套调用与恢复。该栈以数组形式实现，栈顶指针 `top` 控制访问，设计支持最多 **4 层嵌套**，可有效防止用户态 `trapframe` 被多次信号处理覆盖，从而避免嵌套信号异常或 `sigreturn` 恢复错误等问题。此设计增强了信号机制的健壮性，确保 `handler` 能够正确进入和退出，满足 POSIX 信号语义要求。

5. 通过checkpoint截图

```
Running all tests
signaltests starting
test basic1: OK
test basic2: OK
test basic3: OK
handler4 triggered
test basic4: OK
handler5 triggered
handler5 triggered
handler5 triggered
handler5 triggered
handler5 triggered
test basic5: OK
handler6 triggered due to 1
handler6_2 triggered due to 2
test basic6: OK
handler7 triggered due to 1
handler7_2 triggered due to 2
test basic7: OK
```

(据沈老师说basic8测试文件有bug，故未放basic8测试结果)

6. Git提交记录及项目网址

<https://github.com/Zhang-Astrid/CS302-Operating-System-Project-signal.git>

Q	Text or hash	.*	Cc	Branch: signals-SsSsS[signa...	User	Date	Paths	↓	🔍
●	Merge pull request #1 from Zhang-Astrid/signals-SsSsS				origin & signals-hatfail	Chenjun Xu*			Yesterday 19:25
●	checkpoint 1 basic 6,7 passed				origin/signals-SsSsS	oslab			Yesterday 19:08
●	update:passed basic4&5					hatfail			2025/5/18 23:56
●	update: finished handler4					hatfail			2025/5/16 20:46
●	update:passed 2 testcases in 8					hatfail			2025/5/16 19:18
●	improve: export pool[] and add comments					yuki			2025/4/27 22:57